

arkadaşlar selamlar geliyor değil mi sesim Tamamdır bir bir ik dakika daha bekleyelim sonrasında başlayalım tekrardan selamlar şimdi geçen dersimizde ne yapmıştık Prompt engineering kavramıyla başlamıştık bu dersimizde ise prompt engineering'in aslında ileri seviye yani gelişmiş prompt engineering yöntemleriyle devam edeceğimizi söylemiştim ama küçük bir değişiklik yapacağız şimdi reg ile devam edeceğiz çünkü reg yaparken de yine temel seviye prompt engineering yöntemlerimizi kullanıyor olacağız ileri seviyeyi kullanmayacağımız için onu gerçek bir örnek yaparken göstermek istiyorum bu nedenle yine önümüzdeki derslerde Prompt Engineering'in ileri seviye konularından bahsediyor olacağım yani kısaca bugün reg nedir ve neden kullanılır ile devam ediyor olacağız şimdi bir ekran paylaşayım evet şimdi sorumuz şu: Reg nedir ve neden kullanılır R dediğimiz kavram retrieval augmented generation yani bilgi getirme destekli üretim olarak uzun bir şekilde yazabileceğimiz ya da Türkçeleştirebileceğimiz bir kavramdır hemen şöyle bakmak gerekirse buradaki R harfimiz retrieval yani getirmeye tekabül eder a harfimiz augmented yani zenginleştirmeye tekabül eder g harfimiz ise generation yani üretmeye tekabül eder yani retrieval augmented generation şeklinde peki bu REG dediğimiz mevzu ne ola ki R dediğimiz mevzu elemmlerin bilgi tabanlı yanıt üretme kapasitesini arttırmak için kullanılır şöyle düşünebilirsiniz mesela biz şimdi kendi kurumumuz özelinde bir chatbot yapmak istesek ama bu chatbotun sadece bizim kurumumuza özel bilgileri cevaplamasını istesek ne yapacağız Bizim kurum bilgilerimizi bu chatbota bir şekilde verebiliyor olmamız lazım ya da bu eleme bir şekilde verebiliyor olmamız lazım işte bunu nasıl yapabiliriz Deriz ki eleme al deriz bu bilgiler bizim aslında kurumumuzla ilgili özel bilgiler sana bir soru sorulduğu zaman sadece bu bilgilere bakarak cevap vermeni istiyoruz deriz işte bu yapı çok kaba tabiriyle reglandırılabilir tamam şimdi burada diğer bir mevzumuz da şu sabit parametrelili modeller güncel ve özgün veriyle çalışmayı bize ne yapıyor Aslında sağlıyor şimdi güncel ve özgün veri dediğimiz mevzu ne Şuradaki özgün veriyi regarak adlandırabiliriz güncel veri de aslında biraz web search alanına giriyor neden böyle söylüyorum Çünkü eğer chatbotlarımız ya da elmlerimiz bir web search ya da reg yapmasa güncel veriyle çalışamazlar onlar çünkü sadece eğitildikleri veriyebilirler bu nedenle biz reg ya da işte web search gibi kavramlarla ne yapıyoruz Halihazırda elelemimizin cevap vermek istediği güncel konuları ya da özgün veriyi el eleme veriyoruz yani chatbotumuza veriyoruz ve böylece chatbotumuz biz ne istersek onu söyleyebiliyor kullanıldığı alanlardan bahsetmek gerekirse soru ve cevap sistemleri mesela soru ve cevap sistemlerini nasıl düşünebilirsiniz Diyelim ki sizin bir e-ticaret siteniz var ve e-ticaret sitesinde sık sorulan sorularınız var sık sorulan sorular sık sorulan sorular ne olabilir İşte kargom nerede kaldı ya da işte müşteri destek hattı numarası nedir Ya da fiyatlara KDV dahil mi ya da yurt dışına gönderim yapıyor musunuz Sık sorulan sorulardan bahsediyorum eğer bunları bir belge haline getirirseniz ve eleme verirsiniz bunu bu arada kaba tabiriyle söylüyorum yoksa gidip de eleme doğrudan bunu vermeyeceğiz daha detayları anlatmadığım için böyle söylüyorum yani eleme verirsiniz ne yapıyor olacak Bu elem dediğimiz pardon arkadaş yani chatbot sizin sık sorulan sorular dokümanınıza bakarak cevap veriyor olacak böylece bilgi getirme işlemini gerçekleştirmiş olacak tamam bu bizim gerçek

hayattaki kullandığımız en sık kullanılan alanlardan bir tanesi şimdi gelelim doküman analizine doküman analizini de aynı şekilde düşünebilirsiniz elinizde bir tane doküman var tamam mı Bakın şöyle bir doküman var hatta 100 sayfalık bir doküman var çarpı 100 şeklinde yazıyorum bu 100 sayfalık doküman bir tane sözleşme olsun ve siz şirketinizin sözleşmedeki mesela ceza maddelerini öğrenmek istiyorsunuz bunun için ne yapabilirsiniz 100 sayfayı tek okuyabilirsiniz değil mi Ceza maddelerini bulmak için ya da başlıklara göre arama yapabilirsiniz ya da reg altyapısı ile birlikte doküman analizi gerçekleştirebilirsiniz bu ne demek Aslında tanımı çok basit bunun tanımı şöyle yapılabilir bizim bir tane dokümanımız var biz bunu el eleme veriyor olacağız ve sizler bir soru sorduğunuz zaman ya da bizler bir soru sorduğumuz zaman bu soru belki içerisindeki en uygun yerde bulunacak yani sorunun cevabı belki içinden bulunacak ve elelen bu belgeye bakarak cevap veriyor olacak çok kaba tabirle şöyle düşünün ben size bir tane soru sordum siz ne yapıyorsunuz Belgenin içerisinden açıp sorunun cevabını bulup bana cevaplıyorsunuz ya RE işte tam olarak bunu gerçekleştiriyor olacak ve tabii ki chatbotlar tamam şimdi chatbotları şöyle düşünebilirsiniz mesela siz bir tane inputunuz var bunu gönderdiniz bir chatbot'a tamam bu chatbot dediğimiz arkadaş doğrudan cevaplayabilir ya da buradan bir database'den bilgi getirebilir bilgi getirip size bir output return edebilir değil mi İşte biz bu süreci R olarak adlandırıyoruz bunlar bizim günlük hayatta kullandığımız R çalışması yaptığımız sık kullanılan alanlardan bazılarıydı şimdi reg'in bileşenlerini biraz daha açalım çünkü önemli regdeki R harfi retrieval yani getirme bu retrieval dediğimiz mevzu kullanıcının sorgusuna göre vektör veri tabanından benzer dokümanların bulunması kullanıcı bir soru sorar inputu vardır tamam ve bizim bir tane vektör db'miz söz konusu bakın şöyle bu arkadaş vektör db şimdi bu sorula alakalı olan bilgiler ne yapılacak Vektör DP'den getirilecek nereye Bizim el elemimize yani chatbotumuzun merkezine işte biz bu getirme yani bilgiyi getirme olayını retrieval olarak adlandırıyoruz sonrasında augmentation yani A harfimiz yani zenginleştirme getirilen içeriklerin prompt içerisine gömülmesi şimdi bizim halihazırda bir input'umuz var bu input prompt prompt tamam ya da şöyle yapalım şuraya kocaman bir tane prompt yazacağım prompt eşittir ne var içerisinde Bunun aslında bir tane input'umuz var değil mi Kullanıcının sorgusu ardından ne var Bizim bir sistem promptumuz var bu da bizim developer olarak kendi yazmış olduğumuz prompt ardından ne var Bilgi var bu bilgi neyin bilgisi Retrieve edilen bilgi retrie bilgi şeklinde karşımıza çıkıyor bu süreç nasıl oluyor Daha doğrusu bu prompt nasıl oluşuyor Biz input'u soruyoruz gidiyor vektör database'den bir retraving işlemi gerçekleşiyor ve promptumuza yazılıyor yani bilgi getiriliyor e bunların hepsini ben lelmem verdiğim zaman lelm buradaki zenginleştirilmiş prompt içerisindeki aslında metin ile ne yapıyor Bir generation yapıyor yani üretme gerçekleştiriyor yani rateeki k harfinden bahsediyorum buradaki elem aslında inputu doğrultusunda bir cevap üretiyor yani buradaki R A ve G harflerinden G dediğimiz şey bizim daha önceki derslerde öğrendiğimiz elemi kullanarak yapmış olduğumuz soru cevap botu gibi düşünebilirsiniz r ve A ise ne yapıyor halihazırdaki LLM'in kendi bildiklerinden değil dışarıdan bir veri setinden ya da bir vektör database'den ya da farklı bir bilgi kaynağından bilginin getirilip promptun zenginleştirilmesi işlemi şimdi bunu vektör

veri tabanları ve embedding bakış açısıyla bir inceleyelim şimdi bunu yapabilmek için yani şuradaki R'deki R'deki R'yi yapabilmek için bir retrieve işlemine ihtiyacımız var retrieve dediğimiz mevzu neydi Bizim bir vektör database'imiz vardı değil mi Vektör database içerisinden bilginin getirilmesiydi peki buradaki vektör veri tabanları dediğimiz mevzu ne ola ki Şimdi bizim elimizde metinlerimiz var değil mi Bizim bu metinleri depolayabilmemiz için eski usul işte PDF doküm txt kullanmıyoruz bunları bir veri tabanında depolamamız gerekiyor ve vektör formatında depolamamız gerekiyor bu nedenle metinleri vektörlere dönüştürmemiz gerekiyor bunun için de ne yapıyor olacağız Embedding kullanıyor olacağız yani metin bir ok ve vektör işte bu ok eşittir embedding şeklinde karşımıza geliyor yani embedding yaparak elimizdeki metni vektörlere çeviriyoruz sonrasında ne yapıyoruz Vektörleri alıyoruz vektör database'imizin içerisine depoluyoruz aslında şöyle bir soru sormamız gerekiyordu hocam şimdi biz retrieval yaparken yani reg'teki R harfini yaparken input doğrultusunda vektör database'den bilgi getiriyoruz dedik ya bu bilgiyi neye göre getiriyoruz Benzerliğe göre mi Uzaklığa göre mi Yakınlığa göre mi Alakaya göre mi Mantiğa göre mi Neye göre getiriyoruz Aslında burada bağlam karşılaştırması bağlam benzerlik karşılaştırması yapıyoruz bunun için de cosine similarity ya da product gibi neleri yöntemleri kullanabiliyoruz Tamam bunlar bizim bilgi getirirken kullanacağımız benzerlik aslında eee fonksiyonları olarak adlandırabiliriz yani kısaca bilgiyi sormuş olduğumuz soruya benzer olacak şekilde getiriyoruz sonrasında popüler vektör veri tabanlarımızdan bahsetmek gerekirse FIS ve Chrome'a en çok kullanılan veri tabanlarından iki tanesi production aşamasında F ya da Chromay'ı kullanabilirsiniz hani kişisel olarak soracak olursanız biz genellikle FIS'i kullanıyoruz bunlar da vektör veri tabanlarımız şöylece bir özetlemek gerekirse bizim bir metnimiz var ember yaparak vektöre çeviriyorum vektör veri tabanında depoluyorum sonrasında kullanıcı bir soru sorduğu zaman cosine similarity'ye göre vektör veri tabanı içerisinden kullanıcının sormuş olduğu soruya en benzer cevapları alıyorum ve returnle diyorum kaç tane cevap return edeyim 1 mi 2 mi 3 mü 5 mi 500 mü Bu tamamen bize kalmış bu işte bizim context size dediğimiz bir şey vardı ya content size'ımız bununla alakalı ne kadar büyük content size'ımız varsa o kadar çok aslında return edebiliriz bu tamamen bizim elemimizin işlevi ile alakalı bir mevzudu buraya kadar sormak istediğiniz bir şey var mı Tamam sorumuz yok diye anlıyorum özellikle bekliyorum ki hani aramızda eee zaman farkı varsa bir 30 saniye bir dakika bekliyorum zaman farkı oluşabiliyor ya e tamam sorumuz yok güzel devam edelim şimdi akış diyagramını biraz daha detaylandıracağım burası önemli çünkü burada görmüş olduğunuz diyagramın birazdan Python ile kodlamasını yapıyor olacağız şimdi nasıl olur Aslında bunun iki tane boyutu var biz şu an tek bir boyuta bakıyoruz şimdi kullanıcı sorgusu diye bir şey var kullanıcı sorgusu ne olsun İşte merhaba fiyatlara KDV dahil mi Fiyatlara KDV dahil mi Kullanıcı bir tane soru sordu sonra ne yapıyoruz Kullanıcının sorgusunu vektörel hale getiriyoruz vektörleştirme vektörel search yapabilmemiz için vektör database'in içerisinde bulunanlar ile yani bilgiyle kullanıcının sormuş olduğu soru arasında ne olması lazım İkisinin de vektör olması lazım yani sayısallaştırma yapmamız lazım sonra similarity search ile cosine similarity kullanabilirsiniz biz cosine similarity kullanıyor olacağız ya da öklit mesafesi kullanırız

ne yapacağız en alakalı içerikleri getiriyor olacağız yani şurada bir database olacak buradaki soru gelecek buraya soru buradan da bilgi return ediliyor olacak yani retrieve kısmını yapacağız sonra bunu prompt içerisine koyacağız yani augmentation kısmını yapacağız ve generation ile elemen bize bir yanıt üretecek böylece yapay zeka bizim için kullanıcımızın sormuş olduğu soru doğrultusunda bizim bilgi içerisindeki bilgiyi alıyor ve cevap veriyor olacak dediğim gibi bu iki boyutlu bu kullanıcı tarafıydı bakın nerede Şurası burası kullanıcı tarafı bir de bunun database hazırlama tarafı var onun nasıl olduğunu birlikte inceleyelim şimdi ilk olarak sol tarafla başlayacağım bakın data preparation diyor veri hazırlama şu kısım veri hazırlama nasıl bir süreç Elimizde row datamız var bu row data dediğimiz mevzu işte bildiğiniz kitaplar internetteki yazılar işte Wikipedya yazıları ne bileyim işte herhangi bir metin ya da bahsediyorum sonra buradan information extraction yapıyoruz eğer bunlar PDF ise data extraction yapmamız gerekiyor ocr dediğimiz mevzu ne arkadaşlar biliyor musunuz Görüntüler üzerinden bilgi çıkarma yani bir görüntüyü aslında metne çevirme yani aslında burada row dataan extraction yapıyoruz ve bilgi çıkarıyoruz sonrasında elimizde kocaman bir bilgi yığını oluştu bilgi yığını şimdi bunun hepsini alıp da kocaman bir vektör hepsini alıp da hepsini bir anda vektörleştirmeye gerek yok bunu çanklara böleceğiz çanklara bölme şöyle yazalım çanklara bölme bu ne demek biliyor musunuz Elimizdeki yüzlerce sayfa bilgiyi küçük küçük parçalara bölüyoruz mesela örnek vermek gerekirse ben çanklama işlemini genellikle iki şekilde yapıyorum eğer bir soru cevap PDF'ine sahipsem her soruyu ve cevabını bir çank olarak ifade ediyorum mesela soru 1 cevap 1 değil mi Şu an row datayı çiziyorum soru 2 cevap 2 nokta nokta nokta şeklinde şimdi row datadan buraya gelip buraya geldiğim zaman her bir çank yani çank 1 e= soru 1 ve cevabı çankş soru 2 ve cevabı şeklinde parçalara bölüyoruz böylece bilgiler retrieve edilirken sadece ilgili çanklar return ediliyor ve böylece gereksiz çankları return etmemiş yani eleme gereksiz bilgiyle doldurmamış oluyoruz bunu yaparken aynı zamanda başlık başlık da bölebilirsiniz ya da elinizdeki metnin bağlamını koruyacak şekilde kendinize güzel bir çanklama yani parçalama bölme yapabilirsiniz yani bu aşamaya geldiğim zaman elimde küçük metin parçalarım söz konusu tamam şimdi ne yapmam gerekiyor Bunlar hala metin benim bunları embedding ile vektörleştirmem gerekiyor yani embedding yaparak vektörize edeceğiz vektör haline getireceğiz ve alıp bunları vektör database'de kaydediyor olacağız mantık çok basit elimde row data var kocaman bir 100 sayfalık PDF ben bunu ne yapıyorum Önce parçalara bölüyorum sonra edinkile vektörleştiriyorum sonra da vektör database'imizde depoluyoruz bu işin ne kısmı Veri hazırlama kısmı bakın şuraya kadar veri hazırlama gerçek hayatta bunu en başta bir kere yapıyoruz tabii ki sabit bir verimiz varsa yok sabit bir verimiz yoksa veri geldikçe sürekli bunları vektör database'e kaydediyoruz oluşturuyoruz yani vektörize ediyoruz kaydediyoruz vektörize ediyoruz kaydediyoruz ha sabit bir verimiz varsa da en başta yazılmamız çalışırken bir kere tüm veriyi vektörize ediyor vektör database'e kaydediyor sonrasında ne oluyor Sonrasında işte bir zaman sonra geliyor kullanıcımız bir tane soru soruyor şimdi kullanıcının sorusu metin değil mi Benim bu metni doğrudan vektör database'de arama ihtimalim yok benim bu metni ne yapmam gerekiyor Vektörleştirmem sayısallaştırmam gerekiyor e bunu ne yapacağız Şurada

biraz önceki kullandığımız m'in yönteminin aynısını kullanarak sayısallaştırıyor olacağız yani kullanıcının sorusunu sayısallaştırıyorum sonrasında bir cosine similarity ile cosine similarity yani benzerlik ölçütüyle gidiyorum soruya en yakın aslında çankları getiriyorum bakın bu da alakalı data oluyor yüzlerce sayfadan belki de 10 sayfa belki 5 sayfa artık kaç sayfa istiyorsak en alakalı cevapları getiriyoruz bunu en adet veri getir şeklinde yapabilirsiniz n adet ya da benzerlik skoruna göre benzerlik skoruna göre bir trash off daatabilirsiniz cosine similarity 0 la 1 arasında benzerlik skoru üretecektir 0 bunlar hiç birbirine benzemiyor alakasız 1 ise bunlar tamamen birbirine benziyor 0.9 0.85 üzeri ise güzel bir trashol seviyesi olacaktır problemin tipine göre değişmekle birlikte gerçek hayatta biz genellikle 0.9085 civarında kullanıyoruz tamam ya da en adet bilgi getir şeklinde de yapabilirsiniz sonra bu bilgiyi alıyorum eleme veriyorum zaten bir de query'imiz vardı lm'de augmentation'ı yapıyoruz zenginleştirme lm bana bir generate ediyor neyi generate etmiş oluyor Response'u generate etmiş oluyor işte bizim reg dediğimiz sürecin veri hazırlanması ve kullanıcı tarafından işte reg'in kullanılması süreci bu şekilde buraya kadar sormak istediğiniz bir şey var mı Tamam yok diye tahmin ediyorum varsa zaten siz sorunca ben görürüm şimdi devam edelim şimdi neden reg sorusunu cevaplayalım Şimdi gerçek hayatta bizim elemlerimiz benim böyle tatlı tatlı anlattığım gibi mükemmel çalışmıyorlar bazı problemleri var bir kere elemler bir veriyle eğitiliyorlar değil mi Bakın şöyle nerede benim kalemim veri training sonucunda elmem ortaya çıkıyor e güzel veri değişirse ne olacak Değişirse hatta ve hatta şöyle diyelim veri her hafta değişiyor yeni veri geliyor ya da her ay değişiyor ne olacak Her seferinde training mi yapacağız Training kolay bir süreç değil ne olacak Yani sürekli eğitecek halimiz yok ne yapacağız Modeli yeniden eğitmeye gerek olmadığı için reg yapıyor olacağız yani şöyle düşünün bir training modelimiz var regdeki sol taraf neydi Aslında bizim vektör database oluşturma aşamamızdı değil mi Vektör database sağ tarafta kullanıcı kısmıydı kullanıcı yeni veri geldiği zaman veri değişirse sadece vektör database değişiyor olacak onu da değiştirmek çok kolay yeni veri gelirse bir saniyede işte ya da 2 saniyede yeniden hani yeni veriler vektör database'e ekleniyor olacak sağ taraf kullanıcı tarafı ile ilgili hiçbir değişiklik yapılmasına gerek yok böylece verinin çok değiştiği projelerde modelimizi yeniden eğitmek yerine reg ile gayet güzel bir altyapı kurabiliyoruz aynı zamanda reg sayesinde halüsinasyon riskimiz düşüyor halüsinasyon dediğimiz şey bizim dil modelimizin bilmediği bir şey hakkında yorumlar yapması kısaca halüsinasyon görmesi ya da yeni bilgi uydurması bu gerçek hayatta sıklıkla karşılaştığımız bir problem çünkü elemler ben bilmiyorum demezler bildikleri şey doğrultusunda istatistiksel olarak bir şeyler üretmeye çalışırlar bu da dediğim gibi önemli problemlerden bir tanesi çünkü yanlış bilgi üretme söz konusu halüsinasyonla birlikte daha doğrusu reg'le birlikte halüsinasyon riski düşüyor çünkü biz bilgiyi veriyoruz ve bu bilgiye göre cevap ver diyoruz kuruma özel verilerle çalışabilmesi açısından da güzel şimdi bazı kurumlar malumunuz gizliliği ön planda tutuyor tamam bu ne demek Yani kurumun içerisindeki bilgiler kurum dışarısına çıkamaz yani kurum kısaca bizim işte Gemini gibi chat GPT gibi dil modellerini kullanamıyorlar cloud ortamına çıkamıyorlar kurum içerisinde çalışmak gerekiyor ya da kuruma özel verilerle

çalışmak gerekiyor ikisi birbirinden farklı kavramlar kuruma özel verilerle çalışmak gerekirse aynı biraz önceki gibi işte bizim sık sorulan sorularımız kurumdan kuruma göre değişir değil mi E chat GPT ya da Cemilay bunların hepsini bilemeyeceğine göre doğal olarak ne yapmamız gerekiyor Bizim bunları bilgi olarak vermemiz gerekiyor yani kuruma özel verilerle çalışabilme olanağı Rate sayesinde kazanmış oluyoruz ve tabii ki KVKK ve GDPR var yani kişisel verilerin korunumu kanunu bunlara uyumlu çözümler sunuyor çünkü RE kavramını biz genellikle bir bilgi üzerine inşa ettiğimiz için genellikle 10 prem çalışmalarda kullanmayı tercih ediyoruz tabii ki cloud ortamında da kullanılır ama şöyle söylemekte fayda var 10 prem yaptığımız çalışmalarda genellikle R kullanılıyor ve bu da zaten bilginin dışarıya çıkmadığı anlamına geliyor e ama aslında bunun renkle çok bir alakası yok genellikle 10 pem olduğu için tamam şimdi bakalım bir soru vardı eee evet soru yanıtladım yani kullanışlı olması için hangi boyutta çok küçük bir sayfalık bir metin de olabilir 1 milyon sayfalık bir metin de olabilir hiç fark etmez regi kullanma mantığımız dediğim gibi yeni bir bilgi varsa bu bilgiyi chatbotlar bilemeyecektir ya da işte bizim kendi kurumumuza özel bilgiler varsa ya da spesifik bir dokümandan cevap üretmek istiyorsak reg yapacağız reg'in mantığı çok basit benim bir bilgim var git bu bilgiye bak bu dokümandan bu bilgiden bana cevap üret tamam şimdi gelelim birazcık da link chain framework'üne e malumunuz kodlama yapacağız değil mi Kodlama yaparken burada link chain framework'ünü kullanıyor olacağız python tarafına geçiyorum birazdan kodlama yaparken l chain dediğimiz mevzu ne ola ki L chain elenlem tabanlı uygulamaları hızlı geliştirmek için bir framework yani şöyle söylemek gerekirse makine öğrenmesinde ne vardı Sit learn vardı derin öğrenmede ne var Tensor flow var elmler'lerde ne var L chain altyapısı var burada modüler bir yapı sağlıyor bize prompt yazmak retriever tanımlamak memory tanımlamak ve output passing yapmak yani structured output tanımlamak çok kolay oluyor rank agent chatbot tool call gibi senaryolarda hazır altyapı sunuyor bu kısımda REG kısmında ve bundan sonraki agent kısmında biz Lengchain kütüphanesine kullanıyor olacağız bileşenlerinden bahsetmek gerekirse reg özelinde konuşuyorum mesela dokümanları yüklemek için bir tane docum loader isimli fonksiyonu var bununla birlikte pdf txt gibi veriler yükleyeceğiz çunklara ayırmak için bu keyword'ü unutmayın çank çankları ayırmak için bir text splitterı var yani uzun metinleri küçük parçalara bölmek için embedding modelimiz var cümleyi vektöre çevirir vektör store'umuz var vektör store olarak face kroma gibi vektör veri tabanları kullanabiliyoruz biz genellikle F kullanıyoruz dediğim gibi Chrome adama kullanabilirsiniz bu eğitim özelinde ne yapacağımızı hatırlamıyorum birazdan kodlama yaparken şekillenir ama gerekirse ikisinin de örneğini size gösteririm retriver dediğimiz mevzu ise soruya uygun içeriklerin getirilmesi şimdi elem dediğimiz yapı ise zaten onu biliyorsunuz artık geçen dersimizde öğrenmiştik verilen içerikle yanıt üreten bizim yapay zeka modelimiz large language model şimdi rank r chain tasarımından bahsetmek gerekirse bundan bahsetmiştik zaten kullanıcı sorgusu var vektör database'den bilgiyi retrieve ediyorum l chain'ile prompt template yazıyor olacağım elm cevap üretecek ve cevabım benim istediğim formatta mı değil mi diye structured output parcel ile kontrol ediyor olacağım şimdi burada iki tane mevzu söz konusu birincisi cloud

ortamında çalışabiliriz ikincisi de eee kendi onprem lokalimizde çalışabiliriz kodlama yaparken hangisini yapacağımıza karar veririz 10 premde çalışmak da olabilir 10 premde çalışmayı size göstermedim onu gösteriyor olabilirim mesela olamaz sayesinde olama da bir framework bu arada eee CEMO modellerini kolayca entegre edebiliyoruz l chain olama elem sınıfı kullanıyor e kullanmak zorunda değiliz burada yine bir sürü kütüphane söz konusu ve Jason on çıktılar için format Jason kullanılabilir hafif modeller 1 milyar parametrelili modeller ile kendi bilgisayarınıza da çalıştırma yapabilirsiniz şimdi birazcık kod yapalım kod yazalım şunu bir kapatalım sorunuz varsa yine sorabilirsiniz şimdi buraya bir tane RDF folder açacağım şöyle geldim şimdi bir tane genel diye klasör açalım bu yine bizim en temel örneklerimizi içerecek olan klasör olacak buraya bir VS kod açıyorum kod nobşluk şeklinde yazdım vs kod karşımıza çıkacak şunu biraz büyütelim şunu kapatalım bir tane terminal açalım şöyle de sola çekelim tamam şimdi size ilk olarak bir 10 prem örneği göstermek istiyorum 10 çalışmayı da öğrenelim çünkü geçen dersimizde zaten cloud ortamında çalışma gerçekleştirmiştik ilk olarak bir ne yapacağımızı konuşalım yani reg yapacağız da nasıl yapacağımızı bir konuşalım sonrasında adım adım ilerliyoruz olacağız şimdi bir alt çizgi main şeklinde yazıyorum main buraya bir doc string açıyorum ne yapacağız Reg yapacağız reg dediğimiz mevzu iki tane aşamadan oluşuyordu vektör database oluşturma değil mi Oluşturma vektör database kullanarak kullanarak şöyle yapalım kullanarak reg + elm yapıyor olacağız yani reg + elm dediğimiz şey aslında reg + l elm değil regin içerisinde lelm ile cevap üretme elemen ile cevap üretme şeklinde yazmış oldum tamam şimdi vektör database oluşturma işlemini nasıl yapacağız İlk olarak birincisiyle başlayalım ilk olarak yapmamız gereken şey bir tane PDF bulmak pdf bul değil mi Yani aslında bir metne ihtiyacımız var sonrasında PDF dosyasını yükle dosyasını yükle sonrasında PDF dosyasını çanklara böleceğiz pdf dosyasını çanklara böl bu kodlama aşamasında benimle birlikte yazabilirsiniz hem tatlı tatlı yavaş yazıyorum hem de copy paste yapmayacağım bu önemli bir konu yani bazı konular tabii ister istemez hani önem seviyesi çok da yüksek olmayabiliyor bu nedenle bazen copy paste yapıyorum ama bu gerçek hayatta bizim de çok sık kullandığımız sizin de elinizin alışmasını istediğim konulardan bir tanesi bu nedenle benimle birlikte yazabilirsiniz gerekirse sizi beklerim emre ile metinleri sayısallaştıracğız 4 numara emreli metinleri sayısallaştır 5 numaraya geldiğimizde ise vektör veri tabanı oluşturacağız vektör veri tabanı oluştur şeklinde yazıyoruz burada da vektörleri depolayacağız işte yani oluşturacağız ve depolayacağız oluştur ve depola bu işin vektör database oluşturma kısmıydı bir de aslında R kısmı var değil mi Burada ne yapacağız Kullanıcı sorusunu sorar sorusunu sorar sonrasında embedding ile kullanıcı sorusu vektörreze edilir sonrasında retrieve işlemi yapılır retrieve dediğimiz mevzu neydi Vektör database üzerinden retrieve işlemi vektör DB üzerinden bilgi getirme sonrasında augmentation yapılır augmentation bu da bilgi art şeklinde yapabiliriz ve son olarak da generation generation yani aslında elem ile cevap üretme şeklinde yazıyoruz tamam bunlar bizim temelde yapacağımız gayet güzel adımlarımız şimdi bunları yapabilmek için ne yapmamız gerekiyor İlk olarak bir virtual environment kurmamız gerekiyor python and virtual environment virtual environment şeklinde sanal ortamımızın kurulumunu

gerçekleştiriyor tamam sonrasını aktif hale getireceğim virtual environment
back/script back/activate şeklinde aktif hale getik şimdi kütüphane kurulumlarımızı
yapacağız kurulumlar pip install chain rank chain kütüphanesini kuracağız chain
community Blank chain hub PDF PDF ile PDF işleyeceğiz vektör veri tabanı olarak
chroma db kullanacağım bu nedenle buraya chroma dp şeklinde yazıyorum f de
kullanabiliriz fark etmez sentence transformers kullanacağım sentence Transformers
bu da bizim embedink yöntemimiz için gerekli olan kütüphane olacak ve son olarak
da olama framework'ünü kullanıyor olacağım şimdi bunların bir kurulumunu yapalım
getirdim buraya yapıştırdım kurulumları birazcık uzun sürebilir yani uzundan kastım
bir dakika falan sürebilir kurulumlar tamamlansın sonrasında devam ederim
görüyorsunuz hala kurulumlar cevap veriyor şey oluyor f Kurulum hala devam ediyor
biraz uzun sürdü evet kurulumuz tamamlandı şimdi bir de olama kuralım olama
dediğimiz mevzuyu arkadaşlar bir framework olarak tanımlamıştım olama.com'a
gireceksiniz buradan download'a bastığınız zaman Mac Linux ya da Windows için
seçenekler karşımıza çıkıyor e ben Windows kullandığım için Windows için download
diyeceğim şu an bende kurulu ve normal next diye kurulumumu yapıyor olacağım
bunu yaptıktan sonra da şunu yapacaksınız bakın konsolunuzu açacaksınız command
promptunuzu bakın şöyle yapalım kurulum tamamlandıktan sonra buraya olama
yazdığınız anda olamayla ilgili kullanabileceğiniz komutlar karşınıza çıkacak buraya
olama list dediğiniz anda bende daha önceden kurulu olmuş modeller ortaya çıkacak
sizde ise hiçbir şey ortaya çıkmayacak bu modellerden buradaki modeller kısmına
geldiğimiz zaman e istediğinizi indirebiliyorsunuz biz hangisini indireceğiz Google'ın
dil modeli olan Cema 3 bakın şuradaki dil modelini indiriyor olacağız hatta CMA3'ün
hangisini kullanalım 1 milyar parametrelili modelini kullanalım çünkü ya da 4 milyarı
kullanalım ya ya 1 milyar ya 4 milyar ikisinden birini indirebilirsiniz ben bende
hangileri varmış bakayım ben 1 milyar olanı kullanıyor olacağım siz de daha hızlı
inmesi adına 1 milyar olanı indirebilirsiniz 1 milyar olanı indirmek için CMA 3 1
milyara basıyorum bakın burada sonrasında tek yapmamız gereken şey olama şunu
kopyalayıp getirip buraya yapıştırmak olacak enter'a bastığımız anda bu arkadaş şu
an bende çalışacak tabii ki sizde de kurulum başlayacak kurulum başladıktan sonra
sizde de çalışıyor olacak bakın bir çalıştığını görelim bunları ben yine size atayım şunu
indirebilirsiniz diye merhaba sen kimsin diyelim bakın merhaba sen kimsin dediğim
zaman ben Google tarafından eğitilmiş büyük bir dil modeliyim şeklinde cevap verdi
şimdi buraya kadar gelmenizi istiyorum dediğim gibi olamaya gireceksiniz modelinizi
olamayı download edeceksiniz sonrasında cema 3 1 milyar parametrelili olan modeli
download edeceksiniz indirmek için şunu yazmanız yeterli bunu yazdığınız zaman
otomatik indirecektir sonunda da zaten aynı bende görmüş olduğunuz gibi
çalışacaktır nasılsın diyelim Ben iyiyim teşekkür ederim siz nasılsınız şeklinde bir
cevap verdi görmüş olduğunuz gibi tamam şimdi bir mola yapalım siz bu arada bunu
indirin o yüzden aslında şimdiye sarkıttım e molada bir 15 dakikada indirirsiniz diye
düşünüyorum o kadar uzun sürmüyor sonrasında devam ederiz saatimiz 6.13 olmuş
pardon saatimiz kaç bu arada Değil mi 6.53 eee 19 08'de görüşmek üzere evet ne
yaptınız arkadaşlar Kurulumlar tamamlandı mı Ah kurulumlar olmadı herhalde diye
düşünüyorum bir ik dakika daha bekleyelim sonra devam edelim tamamdır bunları

yaptınız diye düşünüyorum kaldığımız yerden devam edebiliriz şimdi gerekli olan kütüphaneleri içeriye aktarmıştık ne yapacağız Kodlama yapacağız aslında bunun için vs kodumuzu açalım gerekli olan kütüphaneleri kurmuştuk pardon şimdi içeriye aktarıyor olacağız roml community'den nokta document loader import pdf loader şeklinde yazıyorum from link chain text splitter splitters splitters import recursive karakter recursive karakter text secretor şu arkadaş neçin için de biraz önceki yazdığımız PDF yüklemek için de PDF yükleme şeklinde yazıyorum şu arkadaşlar çankılara ayırmak için chanlara ayırmak sonrasında from LCE'yi yine community embeddings import hugedings bunu da mink için kullanacağız fromchain bunları da yazıyoruz ama inşallah bir sıkıntı çıkartmaz şu an altları çizili malumunuz şu an tanıyamıyor sistem ama genelde böyle oluyor vector store import chrome vektör db kullanacağım bu da vektör dbmiz from Lin yine community elemlerden import olamayı içeriye aktaracağız fromchain nokta chainlerimizden import retrieval QA'yi içeriye aktaracağım bu da bilgi artı elem sağlıyor olacak bize şimdi buraya kadarki olan kısmı bir çalıştırılım bakalım herhangi bir hata var mı Python 1 main payımızı bir çalıştırılım bir şeyleri yanlış yazmış da olabiliriz varsa düzeltelim yoksa devam edelim yanlış yazmışız neyi yanlış yazmışız Chas import retrieval aslında yanlış yazmamışım ama tamam bunun yeri değişmiştir hemen bunun çözümüne bakalım diyor ki "E Leng chain chain diye bir şey bulamadım." diyor bu eski olduğu için bulamıyor diyor ki bunun çalışması için pip installin clasic de kur diyor sonrasında bu çalışacaktır diyor tamam tekrardan çalıştırılım şuraya da yazmış olalım gene tabii bir de şu kısmı değiştirmek lazım burayı klasiya çevirdim şimdi tekrardan çalıştırılım tamam şu an herhangi bir problemimiz yok şimdi bir numaralı adımla başlayacağız yani PDF'ten veri yükleme adımını gerçekleştiriyor olacağım şimdi loader eş pdf loader aç kapa parantez buraya bir tane PDF'e ihtiyacımız var example nopdf şeklinde diyeceksiniz ki hocam böyle bir şey yok evet yok şimdi bunu birlikte bir bulalım mesela Google Wikipedia gidelim şuna gidelim hatta Türkçesine gidelim tamam şu arkadaşşı alalım şöyle aldım alım tarihçenin şöyle sonuna kadar aldım şunu da alıyor ya onu alsın istemiyorum şu kısmı allah Allah şöyle yapsam nasıl yapacak Tamam copy pasti'i kolay bir şekilde yaparım ben de şuraya geldim bir tane PDF oluşturacağız bir tane Word dosyası oluşturalım example şeklinde sonra bunun içerisine siz de yapabilirsiniz bunu kopyaladım sonrasında geldim tarihçi kopyalayacağım kopyaladım bunu kapattım tarihçiyi kopyaladım sonra bunu bir kaydedelim farklı kaydet diyorum bunun içerisine PDF olarak kaydediyorum görmüş olduğunuz gibi example PDF'imiz karşımıza çıkmış oldu şimdi bunu yükleyeceğiz ve bunun içerisinde artık Google'ile ilgili bilgilerimiz var bu bilgilere erişmeye çalışacağız example PDF'imizi burada görebiliyorsunuz sonrasında pages e= noder n yükle split yap aç kapa parantez diyoruz şimdi metni parçalara ayıracağız metni parçalara ayır splitter cursive karakter text splitter chun size'ı tanımlayacağım önce chun size dediğimiz yapı neydi Bizim şanklarımızın boyutunu belirleyen yapı 200 olsun yani her parçanın maksimum karakter sayısı her parçanın maksimum karakter sayısı chun overlap ise overlap çunlar arasında bağlamın kaybolmaması için gerekli olan overlap sayısı parçalar arasında üst üste binme binen karakter sayısı karakter sayısı yani iki tane çankın kesişim kümesi ne olacak 50 karakter olacak bu 50 karakter her iki çunkta da oluyor olacak sonrasında

chunks e= splitter no split document pages şeklinde yazıyorum kaydettim buraya kadarki olan kısmı çalıştırdım bakalım bir problem bir hata bir şey var mı Tamam herhangi bir problemimiz gözüküyor devam edelim şimdi embedding modeli ile vektör haline getirme işlemini yapacağız embedding modeli ile vektör haline getirme şeklinde yazıyorum şimdi bunun için ne yapacağım Hugging face'ten bir tane embedding modeli seçmemiz gerekiyor gelelim buraya diyelim ki hugging face embedding models diyelim text embedding models diye karşımıza çıkıyor buraya girdiğimiz zaman kullanabileceğimiz bir sürü embedding modeliyile karşılaşıyor olacağız bir model seçerken genellikle tabii Türkçe mi değil mi ona bakmak lazım onun dışında da en çok kullanılan genellikle iyidir onu seçebilirsiniz aklınızda olsun buna neden girmedi Gir bakalım bir eee bu değil ya bunu gelmek istemiyorum şu modeller hugging face tamam şuraya embedding yazalım bakın bunlar bizim embedding modellerimiz burada farklı farklı embedding modelleri var bunlardan dediğim gibi sıralama yapabilirsiniz trending de en çok like alana bakabilirsiniz değil mi Ya da en çok indirene bakabilirsiniz mesela burada biz hangisini kullanacağız Şöyle bir bakmak gerekirse e buradan şu an gözüm seçmedi ha bakın şu GEMA 300 milyonu biz gerçek hayatta kullanıyoruz aklınızda olsun tabii tam olarak bu değil de Cema'nın 300 milyon olanlığını falan kullanıyoruz onun dışında bunu kapatabiliriz kullanacağımız modelin ismini yazacağım şöyle yazalım embedding model hugging face embedding model name'imiz eşittir şu arkadaş olacak şu şekilde yazmış oldum sonrasında ne yapacağız Vektör DB eş kroma nokta from document şeklinde yazıyorum docum chants şeklinde yazacağım bu embedding yapılacak metin parçaları yapılacak metin parçaları sonra embedding için embedding modelimizi kullanacağız embedding embedding model şeklinde sonra purest directory yani modelimizi daha doğrusu database'imizi kaydedeceği yer R chroma dp bu kaydedilecek dizimiz olacak güzel buraya kadarki olan kısmı bir çalıştıralım bakalım bir problem eksik gedik bir şey var mı Siz bunu yaptığınız zaman bu model sizde yoksa ki yoktur ne olacak İndirecek bende var diye şu an bunu bir indirmez diyeceğim gerçi de indiriyor da olabilir silmiş de olabilirim hatırlamıyorum yoksa indirecek varsa zaten indirmeden devam ediyor olacağız tamam indirme işlemimiz gerçekleşti şimdi ne yapacağız Aslında vektör database'imizi görmüş olduğunuz gibi oluşturduk chrome SQL 3 şeklinde karşımıza çıkıyor eding yapılan textlerimizi görebilirsiniz bakın çanklarımızı görebilirsiniz toplamda 27 tane çankımız varmış şeklinde karşımıza çıktı güzel birinci aşamayı tamamladık şimdi ikinci aşamayla devam edeceğiz yani reg yapacağız bunun için şöyle bir reg yazalım sonra lelm tanımlama yapacağız lm tanımla lm eş olamaş kafa parantez model eş cema 3 4B bunu nasıl anlıyorduk Bu olduğunu comment front prontumuzu açıyorduk olamaz list yazıyorduk bilgisayarımızda kurulu olan modeller karşımıza çıkıyordu siz Cema 31B'yi indirdiniz problem yok ben 4B'yi kullanacağım şimdi fark etmez daha doğrusu fark eder de eğitim için fark etmez tabii ki daha çok kapasiteli olan model daha güzel cevap verecektir temperature parametremiz vardı buna 0.1 diyeceğiz yani garantici yaklaşım sergiliyor olacağız reg yaparken bunu düşük tutuyorduk doğaşılama yapmasın garantici olsun diye rak chain oluşturacağım retrievable q from chain type len len retriever vector db retriever Search type similarity search olacak bu similarity keyword'ünün karşılığı cosine similarity search

keyword argümanlarımız ise $K = 3$ olsun yani tane belgeyi bize retrieve edecek belge sayısı şeklinde yazmış olacağım şimdi bunu çalıştıracağız ama bunu bir while döngüsünde yapalım while trueen bir query alalım yani kullanıcı sorusu input aşk kapa parantez soru şeklinde yazıyorum result ise ne olacak Regin no invoke aç kapa parantez query yani reg'e neyi gönderiyoruz Query'yi gönderiyoruz print aç kapa parantez f string cevap yine bir aşk kapa parantez result'ın içerisinde bulunan result şeklinde karşımıza çıkıyor olacak şimdi bunu kaydettim temizledim çalıştırdım sonrasında diyelim ki bir merhaba diyelim değil mi Merhaba sen kimsin diyelim merhaba sen kimsin diyorum bakalım ne diyecek sonra da daha mantıklı bir soru soralım aha bilmiyorum demiş tamam şimdi sorumuzu soralım açalım şuradan diyelim ki mesela tarihi ile ilgili bir şey soralım ne yazmıştık Google'a ait diğer siteler nedir Google'a ait diğer siteler nelerdir Bakalım cevaplayabilecek mi Evet bakın YouTube Blogger Orkut ve çok sayıda uluslararası Google sitesi şeklinde şurayı cevapladı ben de zaten burayı sormuştum google ne zaman kuruldu diyelim bakın şu kuruldu a bilemedi okey şöyle diyelim google'ı Google kimler tarafından kurulan bir şirkettir diyelim bakalım bilebilecek mi Allah Allah bilemiyor değişik tamam olabilir yani burada pek çok şey var bilememesinin nedeni seçtiğimiz embellishing yöntemi bir kere Türkçeye uygun değil o nedenle oluyor olabilir eee çanlamada bir problem yaşamış olabilir miyiz diye düşünüyorum biraz çankları uzun tutalım 3 değil de şunu arttıralım şunu bir 500'e çekelim tamam 500 karakter hatta 1000 karakter olsun 1000 250 olsun sonra bunu durduralım şu arkadaşı bir silelim sonrasında tekrardan çalıştıralım baştan çantlarımızı oluşturunuz diyelim ki Google ne zaman ve kimler tarafından kuruldu tarihi yanlış çekti 96 yazmıyor muydu 98 yazıyor mu bir yerlerde 96 96 Niye 98 dedi 1998 ha şurayı almış özel bir şirket olarak kurulduğu kısmı şurada almış şimdi burada isterseniz şunu yapabilirsiniz doğru çalışıp çalışmadığını görmek için kendiniz bir tane metin oluşturun mesela example diyin tamam mı Sonra bunu alın şuraya nereden bizim arkadaşlar Lerry Sergi ve diyelim ki hatta şöyle Lerry Sergi ve Kaan Can Yılmaz Yılmaz diyelim böylece buradan yaptığını da görmüş olursunuz kaydettim farklı kaydet dedim yine buradan example'ımızı seçtik kaydet diyorum pardon PDF seçemedim evet kaydettim şimdi buna girdiğimiz zaman bakın ne yazıyor ve Kaan Can Yılmaz tarafından diyor şu arkadaşı siliyorum şunu da durduruyorum zaten şey hatası verdi tekrardan çalıştırıyorum google ne zaman ve kimler tarafından kuruldu şeklinde soruyoruz bu sefer işte buradaki Kaan Can Yılmaz'ı da yazdırıyor olmasını bekliyoruz Erik Schmit yazdı kiren Yılmaz niye yazmadı Allah Allah şuradaki H inanmadı çünkü o da tamam yani şöyle şu yüzden yazmadı çünkü burada çok fazla Lerry yazıyor ya eee sadece buraya yazmak yetmedi buradaki akışı da sağlamak gerekiyordu bu nedenle yazmadı tamam sıkıntı değil şimdi farklı bir örnek yapalım burayla ilgili sormak istediğiniz bir şey var mı İlk olarak bunu sorayım sonrasında yine farklı bir örnekle devam ediyor olacağız tamamdır o zaman size başka bir şey göstereceğim eee farklı bir yapı aslında r kısmını bitirdik ama biraz da web uygulamaları kısmı ile ilgili örnek vermek istiyorum bu önemli bir mevzu bu nedenle birazcık da bunu web uygulaması haline çevirelim şimdi burada yapmamız gereken ilk şey ne olacak Burada yazmış olduğumuz kodu ikiye böleceğiz neden Çünkü normal şartlar altında vektör database oluşturmayla RE tarafını aynı anda

yapmıyoruz değil mi Yani bir tanesi bizim yazılımımızın süreçlerinden ilk parçası öbürü de zaten sürekli son kullanıcının kullandığı kısım bu nedenle ilk olarak bunu ikiye bölelim diyelim ki e create vectordb.py dosyası sonrasında da eee şöyle yapalım buna main pay dosyası diyeceğim aslında bir main payımız vardı şimdi bu gerçek main pay gibi bir şey olmuş olacak şu arkadaş bir ikiye böldürelim aslında bunu şöyle yapalım şuraya kadarki olan kısmı copy paste yapacağız şu kısmı şurayı da düzeltelim sadece burada vektör db kısmı oluyor olacak tamam şuradaki main pie dosyasını da okuyacağız şunu şöyle getirip yapalım şöyle yapacağım main şeklinde çalıştırıyor olacağız merhaba Google ne zaman kuruldu diyor ki Ocak 96'da Kaliforniya'daki şey tarafından kuruldu diyor baskın da farklı cevap veriyor bu farklı cevap veriyor olması bir kere verinin kalitesi olmasından kaynaklı ve modelimizin tabii ki çok güçlü bir model olmamasından kaynaklanıyor genellikle REG'de en önemli faktör verinin kalitesidir burada biraz farklı şeyler yazdığı için tarih kısmında ve giriş kısmında bu nedenle farklı cevaplar verebiliyor ama önemli olan şu an dokümandan bir getiriyor olması şimdi yapacağımız şey şunu durduracağız ve bunu bir fast API'ye çeviriyor olacağız pes API dediğimiz şey bizim geliştirdiğimiz algoritmalarımızı bir web servisi üzerinden çalıştırmamıza olanak sağlayan bir yapı yani sizlerle web servisi üzerinden çalışan bir tane endpoint oluşturuyor olacağız onun ismi de şöyle olsun main alt çizgi api.p dosyası şeklinde yazalım şimdi buradaki amacımız ne yapacağız Aslında vektör database'imizi oluşturmuştuk değil mi Sonrasında bunu kullanmıştık ve artık bunu bir web servis üzerinden son kullanıcıya sunuyor olacağız bunun için şöyle güzelce bir başlayalım en tepeden ilk olarak ne yapacağımızı bir konuşalım sonra da adım adım kodlarımız zaten drake ve fasti ile fasti ile vektör db kullanarak ne yapacağız Cevap üretme şeklinde yazıyorum buradaki amacımız daha önce oluşturulduğu oluşturulan kromovektör veri setini Fest AP üzerinden kullanacağız şöyle güzel yazalım kötü yazınca için rahat etmiyor daha önce oluşturulan promal vektör veri tabanını fast api üzerinden kullanalım ardından kullanıcı AP'e soru gönderecek kullanıcı API'ye soru gönderir sistem PDF içerisinden cevap verir şimdi gerekli adımlarımız ne olacak İlk olarak kütüphaneleri içeri aktarmakla başlayacağız içeri aktar tabii burada ekstra kurulum da yapacağız vestap uygulaması oluştur request ve response modellerini tanımla modellerini tanımla embedding modelini tanımla daha önce oluşturulan kroma vektör dbyi yükle olama elenen modelini tanımla sonrasında retrieve zincirini oluştur zcireme oluştur soru cevap end pointini yaz end pointini tanımla sonrasında da aşka da bir şey yok bunları yapıyor olacağız kurulumlarımız ne olacak Zaten pip install ile kurulumlarımızı yapmıştık bunun dışında fast kuracağız unicorn kuracağız ardından şöyle bakalım başka kurmamız gereken bir şey var mı Başka bir şey kurmamıza gerek yok bu ikisini kuruyor olsak yeterli yani kurmamıza gerek var da zaten kurmuş süper şimdi gerekli kütüphaneleri içeri aktarmakla başlayalım gerekli kütüphaneleri içeri aktar bu arkadaş bir numara olacak from PI import api http exception from pantic import base modu field şeklinde yazacağız from chain community şuradan geçen derste yazdıklarımızı copy paste yapabiliriz çünkü aynılarını yazıyor olacağım farklı bir şey yazmayacağım bu nedenle vakit kaybetmeye gerek yok şimdi fast API uygulamamızı oluşturalım fast API uygulaması oluştur f = fasti olacak title'ımız olacak rest API servisi bir description olacak ne olacak

description PDF içeriğine dayalı soru cevap yapan basit R API servicei olacak şimdi bunu nasıl çalıştıracam Bunu oluşturduktan sonra bunu çalıştırmak için şuraya bir tanesi read me yazayım re'nin içerisine şu komutu yazacağız ubicorn ubicorn bizim sunucumuz ubicorn mail app no daha doğrusu 2 bunu app'te çalıştıracam tire reload yani bu ne demek Kaydettikçe kodumda bir değişiklik yaptıkça bu arkadaş çalışıyor olacak ha main API yazmışız çok kötü yazmışım ya ap şöyle kaydedelim şunu kopyalayalım getirelim buraya yapıştıralım hah şimdi çalışıyor oluyor lokalimizde çalışıyor 8.000 boardunda bunu açtığınız zaman karşınıza şöyle boş bir sayfa çıkar ve hata verir buraya slashhc yazarsanız swagger açılır ve swagerda yine boş bir ekran karşınıza çıkacaktır hadi çocuğum çalış bakın bizim biraz önce yazmış olduğumuz reg fast'a çıktı şu an için herhangi bir endpoint eklemedik servisimizin içi boş birazdan yine end point ekleyerek devam ediyor olacağız tamam şimdi buraya kadarki olan kısmı şöyle sizin için de açayım burayı sizler de yazın eee şimdi bir mola verelim sonrasına dönüp devam edelim kodları paylaşacağım evet 8 06'da görüşmek üzere diyelim eğer yetişemediyseniz eee bu kısımdan devam edebilirsiniz zaten şurayı yazdım burası önemli şimdi zaten kodların çoğunu baştan yazıyor olacağız paylaşım kodları ondan sonra görüşmek üzere selamlar tekrardan kaldığımız yerden devam edebiliriz burada bir tane sorumuz var bu yaptığımız klasik R yani bu ne demek Ya klasik R demek biz daha agentic kavramını görmedik bu nedenle agentic tarafına çıkmayacağız gerçi agentic'ten bahsetmeyeceğim ama agentic dediğimiz kavram bu sistemin aslında otonomluk karar verme ve etkileşim yeteneği yani agentic reg bilgiyi arayan değerlendiren ve süreci yöneten bir tane ajan gibi çalışıyor ya bir ajan çalışıyor bu ajan gerekirse reg yapıyor ve hani bilgiyi arayan bulamazsa tekrar arayan yani düşünen karar veren etkileşim yapabilen bir ajanın reg yaptığını düşünebilirsin bizimki burada normal klasik rankden bahsediyorum tamam şimdi nerede kalmıştık Şuradan devam edelim fast API'mızı oluşturmuştuk şunu bir durduracağım çünkü boşuna çalışmasın şimdi 3 numarayla devam ediyor olacağız yani request ve response modellerini tanımlayacağız response modellerini tanımla bir dakika nerede kalmıştık Şimdi request ve response modellerini tanımlıyor olacağız bunun için class yazacağım class question request a kapa parantez base modu question yani kullanıcının soracağı soru string olacak bir field olacak bir şey yazmamıza gerek yok yapabiliriz Sonra class source documents r mode source string olacak page integer ya da string olabilir şöyle yapalım ve tabii ki content preview string olacak son olarak da class answer response yani cevabımız yine base model'dan inherit edeceğiz question'ımız var bir string olacak bir cevabımız var string olacak source list a kapa parantez source dokümanları içeren bir listeden bahsediyoruz şimdi embedding modelimizi tanımlayacağız embedding modeli tanımla şeklinde yazıyorum embedding model hugging face embedding aç kapa parantez model name ne yapmıştık bunu Şurada yapmıştık bakın şu arkadaş almıştık tamam aynı şekilde yine bunu alıyor olacağım buraya getiriyorum yapıştırıyorum sonra daha önce oluşturduğumuz Chrome'a DB'yi yükleyeceğiz daha önce oluşturulan Chroma DB'yi yükle diyorum vektör DB kroma açkafa parantez persist direktörümüz vardı roma DB şeklinde yazıyorum embedding function embedding modelimizi yazıyor olacağız sonra olama modelimizi tanımlayacağız olama modeli tanımla adım adım ilerliyoruz aslında lelm

eşittir olama aç kapa parantez modelimiz ne olacak CMA 3 4B temperature değerimiz ise yine 0.1 yani küçük bir değer olacak sonra retal zincirimizi kuracağız vel quick no from pro chain type Inmş In retriever vektör diye bir as retriever search type'ımız vardı cos similarity'yi kullanıyorduk bunu belirtmek için similarity yazmamız yeterliydi search keyword argümanımız aşkı parantez k nokta üst üste 3 şeklinde yazabiliyoruz source docum şeklinde yazıyorum şimdi bu aşamaya kadar yaptıklarımızı çalıştırmadan önce bir tane sağlık control end point yazacağım he check olarak düşünebilirsiniz bunu appget slash olacak definition home aç kapa parantez return message iki nokta üst üste reci çalışıyor kaydettim geldim servisimi çalıştırdım şimdi Swager üzerinden bu servisimizi göreceğiz yani help checkimizi sonrasında da aslında ne yapacağız soru cevap için gerekli olan end pointin'timizi oluşturmakta devam ediyor olacağız şu bir yapsın şimdi şuradan şeyi açalım bakın R servisi çalışıyor şeklinde yazımız karşımıza çıkmış oldu swager açtığımız zaman da yazmış olduğumuz bir tane end point'timiz var buna trade out diyorum execute diyorum ve karşımıza ne çıkıyor R service'i çalışıyor tamam güzel şimdi asıl end point'timizi yazıyor olacağız şunu yine kapatacağım çok fazla kasmaşın diye şimdi soru cevap endpint'imizi yazmak için şöyle bir comment atalım app.post slash da çalışsın response modelimiz eş answer response olacak definition ask question request question request olacak sırayla başlayacağız bir kere kullanıcıdan gelen bir soruyu alıyor olacağız kullanıcıdan gelen soruyu al şeklinde yazıyor olacağım query request no question olacak bu neden böyle Question answerımız vardı değil mi Request atadık bunu question answerın içerisinde bakın burada question diye yazmıştık nok question dediğim anda kullanıcının query'sini alıyoruz sonra reg zincirine soruyu göndereceğiz result e= re chain noin invoke aşk kapa parantez query query şeklinde bu da neden böyle Bakın burada nerede tanımlamıştık bunu Bunu bir yerde tanımlamış olmamız gerekiyor muydu Bakalım tamam şu an için bir yerde tanımlamış olmamıza gerek yokmuş böyle durabilir sonrasında kaynak belgeleri de göstermek isteriz kaynak vermek adına düzenli bir listeye çevirelim sources eş aşkı parantez liste for doküman in result aşk parantez source docum source eş doküman aç kapa parantez metadata source virgül source gelecek yok gelmiyorsa da bilinmiyor şeklinde yazıyor olacağız page'i de çekeceğiz doküman metadata get aç par bilinmiyor şeklinde yazıyorum content preview page content çok uzunsa ilk 300 alsın bizi uğraştırmamasın replace slash and new satırları boşlukla değiştirsin bunlar güzel gözüksün diye sources aşkap parantez bunun içerisine ne ekleyeceğiz Source document ekleyeceğiz bakın şurada yapmıştık source document diye sourcesin içerisine source docum aşkapa parantez source eş source şeklinde yazıyor olacağım page eş page şeklinde yazıyor olacağız burası aslında çıkan sonuçları görselleştirmek için content preview eş content preview şeklinde yazıyor olacağız sonrasında API cevabını return edelim cevabını return evet return answer API daha doğrusu answer response e bizim zaten bir question'ımız var query bir cevabımız var result result sources var source şeklinde yazacağız trayımızın exception'ını atalım accept exception aslında yazıyorum eğer exception'a düşerse raise http exception çıkartacağız statü cod genel hata kodu nedir 500 de 500 verelim detail hata oluştu diyelim hata oluştu güzel şimdi kaydettim geldim aşağıdan yukarıya çalıştırdım bakalım eksik yedik bir şey hata mata bir şey var mı Şuradan bir

açalım swagerımıza gelelim bunu açalım trade out diyelim merhaba Google ne zaman kuruldu Execute diyelim bakalım hata varsa aşağıda zaten gözükecektir yoksa da cevabımız karşımıza gelecek bakın merhaba google ne zaman kuruldu dedik google 96 Ocak ayında Kaliforniya'da kuruldu dedi kullandığımız dokümanlarda bu PDF içerisinde bu bilgiyi kullandım bu içerisinde bu bilgiye ve bu PDF içerisinde bu bilgiyi kullandım şeklinde karşımıza çıkıyor e neden 3 tane Çünkü biz 3 tane alalım demiştik çankları hatırlarsanız bu nedenle karşımıza aynı PDF içerisinde 3 tane dokümanı aldığını ve çıkarttığını görebiliyoruz tamam buraya kadar sormak istediğimiz bir şey var mı Tamamdır o zaman şimdi bir de ne yapacağız Client tarafını yapıyor olacağız size bunun client tarafını da göstereceğim sonrasında da bunu bir güzel arayüz oturtuyor olacağız şimdi client tarafında yapacağımız şey aslında oluşturduğumuz servise ite kadar kodları yazıyor olacağız bunun için hemen şöyle başlayalım request kütüphanesini içeriye aktaracağım şöyle yapalım sonrasında bir API URL tanımlayacağız nereye stek atacağız Buraya da var şunu alalım hatta slash ask şeklinde yazabiliriz ask and pointerine istek atacağız quest input alacağız bunu terminalden alalım pdf hakkında ya da soru sorun soru 2 üst üstü olsun sonrasında postimize bir payload tanımlamamız lazım point bir question'ımızın olması gerekiyor o da zaten eşittir question şeklinde karşımıza çıkacak tabii ki dictionary olduğu için burası eşittir değil nokta olacak sonra try response eş request atacağım bir post isteği olacak apıllime Jason Pay oldu mu Sonrasında data eş response nokta jonzacağız bu arkadaş bize responsumuzu pars edecek ve bir dataya çevirecek sonrasında print data dediğimiz zaman gelen cevabı yazdırmış olacağız train acceptini yazdığımız zamansa accept exception as e diyelim print e dediğimiz zaman çok basit bir tane client yazmış oluyoruz şimdi sunucumuz bakın burada hala çalışıyor sunucuyu kapatmadım görmüş olduğunuz gibi yeni bir tane terminal açacağım bakın UVCorn burada duruyor yeni bir tane terminal açtım buraya Python client file dosyamı çalıştırıyorum request kütüphanesini bulamadım diyor pip inst request yapalım kütüphanemizin kurulumunu gerçekleştirelim tamam şimdi tekrardan çalıştıralım file dosyamızı neyini bulamadım request kurduk işte pardon sonunda S var hatta çok pardon şurada S olacak zaten diyorum ben de neden şey yapıyor şimdi oldu bakın sorunuzu sorun diyor google ne zaman kuruldu Requesti şurada yanlış yazdık request request s olacak kaydettim tekrardan çalıştırdım google ne zaman kuruldu Bakın bunu sorduğumda artık buraya siteğimizin düştüğünü göreceğiz eee çok sorgu gönderdik diye böyle bir şey yaptı bir de 8.000 olacak şurası 800 değil tekrardan şu arkadaş bir durduralım sunucumuzu bir durduralım tekrardan sunucumuzu bir çalıştıralım tekrardan client tarafındaki terminalimize gelelim google ne zaman kuruldu şeklinde sorumuzu gönderelim buraya geldiğimiz zaman isteğimizin geldiğini birazdan göreceksiniz bismillah post his site isi geldi burada da cevabımız döndü 96 yılında Kaliforniya'da şeklinde karşımıza çıkmış oldu süper tamam şimdi buraya kadar sormak istediğimiz bir şey var mı Yani client tarafıyla ilgilen Tamamdır o zaman şimdi yapacağımız son bir adım kaldı o da Streamlad kütüphanesiyle bunu güzel bir uye çevirmek şimdi burada yapacağımız şey dediğim gibi Streamad kütüphanesini kullanarak güzel bir arayüz yapacağız bunun için Stream kütüphanesini önce bir kuralım pip install streamet diyorum bir de request'i de copy

yapmış oldum burada streamet kütüphanesini kuruyor olacağız evet kurulumuz tamamlandı şimdi gerekli kütüphaneleri içeriye aktaralım import Streamad SST diyorum streamad kütüphanesi bizim web arayüzü oluşturmak için kullandığımız güzel bir Python kütüphanesi demo ürünleri çıkartmak için etkili production'da çok kullanmayı tercih etmiyoruz ama şu an bizim için gayet güzel bir işlevi olacak bir API URL'imiz olacak şuradan copy paste yapabiliriz st no set page config aşk kapa parantez page title eş re chat olacak başka da şu an için bir şeye ihtiyacımız yok st.title bu da sayfamızın başlığı öbürü aslında browserdaki e yukarıda olan şey var ya onun başlığı PDF R chat şeklinde yazıyor olacağız sonrasında bir tane alt başlık yapalım st.caption caption başka parantez PDF içeriğine içeriğine göre soru cevap yapan basit bir arayüz ya da basit bir chat diyelim sonrasında if messages not in st sessionstate.messages yani boş bir session state tutuyoruz boş bir liste for message diyelim session state messages with st.c chat message a kapa parantez roll şeklinde yazıyorum message aşk kapa parantez content şeklinde de yazıyoruz şimdi user questionine alacağız kullanıcı sorusu st no chat input diye bir input field yapacağım soru sorun eğer user sorumuz varsa st.session state messages no append aç kapa parantez şöyle yapalım r no üst üste user content user question şeklinde yazıyoruz sonra with st chat messages bu user olacak bir user yapacağız bir asistan yapacağız userın taraflarına st.R ile user'ın sorusunu yazıyor olacağız e tahmin edebileceğiniz gibi asistan tarafına da asistanın vermiş olduğu cevabı ekleyeceğiz şunu copyte yapayım o yüzden user değil buraya asistan yazalım şimdi asistan cevap verirken biraz uzun sürebilir buraya bir tane spinner ekleyelim cevap hazırlanıyor şeklinde şimdi altına bir try yazacağız bir response return edecek bize aslında şu client kısmında yaptığımız şeyin aynısını yapabiliriz bakın şurayı çünkü bunu zaten bu yüzden yazdık direkt şuradan trayı alıyorum tamam mı Şöyle yapalım response payload'umuzun olması gerekiyor payload'u aslında şöyle tanımlayabiliriz şunu silip question biraz önce tanımladığımızın aynısını tanımlıyoruz ama sadece payload değişkeni isminde de bu şekilde tanımlıyoruz datamız var bir de datamızın içerisinde cevabımız var ener çekelim tamam sonra bunu ekrana yazdıralım artık prinde ihtiyacımız yok ens şeklinde sd session messages en sona da buradaki session'ın message kısmına ekliyorum asistan olarak ve içeriğine de contentine de cevap ekleyeceğim yani content eşittir cevap gibi düşünebilirsiniz güzel şimdi bunu çalıştırmamız lazım bunu çalıştırabilmek için yapmamız gereken şey yine farklı bir çalıştırma yöntemi var onu da sizin için şuraya yazayım ratmin içerisine streamlit raml.byp böyleydi diye hatırlıyorum yanlışsa şimdi bakarız chatten bunu çalıştırdığım zaman zaten sunucum çalışıyor mu Zaten sunucum çalışıyor onunla ilgili bir sıkıntı yok bunu da çalıştırdığımız zaman eee bir hata verdi hatayı düzeltelim e set page config yanlış yazmışsın diyor bakalım set page config neredesin Set page config he title'ı şu ts olacak tekrardan kaydettim bunu refresh dedim bakın PDF R chat çıktı soralım google e zaman gruptu şeklinde yazdım cevap hazırlanıyor diyor şuradan Unicorn'a bakacak olursak birazdan şunu yüklüyor hala biraz uzun sürebilir cevap gelmesi daha yeni ayağa kalkıyor yani kaydettik ya tamam şimdi birazdan bu istek düşecektir buraya sonra onu alıp işlemeye devam edecek evet cevabımız görmüş olduğunuz gibi karşımıza çıktı 96 yılında Kaliforniya'da kurulmuştur şeklinde gayet

güzel bir cevabımızı almış oldu böylece ne yaptık Stream de güzel bir web arayüzü de inşa etmiş olduk şimdi başka bir sorunuz var mı başka Kimler kurdu bu şirketi diyelim sizin başka bir sorunuz var mı Sorunuz varsa onu cevaplayalım yoksa da dersimizi bitiriyor olacağız bugün bayağı bir şey yaptık yani baştan sona hem Rag'in teorisini öğrendik hem basit kodlamaya başladık hem de görmüş olduğunuz gibi hem backend tarafında bir servis inşa ettik hem de frontend tarafında güzel bir web arayüzü oluşturduk ve sonuç olarak da cevabımızı da böyle vermiş olduk eğer sorunuz yoksa herkese iyi akşamlar arkadaşlar bir sonraki dersimiz yanlış hatırlamıyorsam ne zaman Yarın değil zaten tatil var cumartesiydi diye hatırlıyorum ya da pazardır hafta sonu görüşmek üzere ben teşekkür ederim iyi akşamlar